

Índice

1. ¿Qué es Git?	2
2. ¿Para qué sirve el comando git init?	2
3. ¿Qué es una rama en Git?	2
4. ¿Cómo saber en cuál rama estoy trabajando?	2
5. ¿Quién creó Git?	3
6. ¿Cuáles son los comandos esenciales de Git?	3
7. ¿Qué es Git Flow?	4
8. ¿Qué es el desarrollo basado en trunk (Trunk Based Development)?	4

1. ¿Qué es Git?

Git es un sistema de control de versiones distribuido, creado para gestionar proyectos de software de cualquier tamaño con velocidad y eficiencia. Permite registrar el historial de cambios de un conjunto de archivos a lo largo del tiempo, de modo que sea posible volver a versiones anteriores, comparar cambios, identificar quién modificó qué y cuándo, y trabajar de forma colaborativa sin sobrescribir el trabajo de otras personas.

A diferencia de los sistemas de control de versiones centralizados (como SVN o CVS), en Git cada colaborador posee una copia completa del repositorio, incluyendo todo su historial. Esto significa que la mayoría de las operaciones (crear ramas, revisar el historial, hacer commits) se realizan de forma local y no requieren conexión constante a un servidor central, lo cual hace que Git sea muy rápido y resiliente ante fallos de red o de servidor.

2. ¿Para qué sirve el comando git init?

El comando **git init** se utiliza para crear un nuevo repositorio de Git en un directorio. Al ejecutarlo, Git crea una subcarpeta oculta llamada **.git**, que contiene toda la estructura necesaria para el control de versiones: la base de datos de objetos, las referencias a ramas, la configuración del repositorio y el índice (staging area).

En la práctica, este comando es el primer paso para comenzar a versionar un proyecto que todavía no está bajo control de versiones. Una vez inicializado el repositorio, ya es posible agregar archivos al área de preparación (**git add**) y registrar los primeros cambios mediante **git commit**. Es importante señalar que git init no crea copias remotas ni sincroniza nada por sí solo; únicamente prepara el repositorio local.

3. ¿Qué es una rama en Git?

Una rama (branch) es, en esencia, un puntero móvil e independiente hacia un commit específico dentro del historial del proyecto. Las ramas permiten que distintas líneas de desarrollo avancen de forma paralela y aislada, sin que los cambios de una afecten directamente a las demás, hasta que se decida unificarlas mediante una fusión (merge).

Por defecto, todo repositorio de Git comienza con una rama principal (tradicionalmente llamada **master** o, en convenciones más recientes, **main**). A partir de ella se pueden crear nuevas ramas para desarrollar funcionalidades, corregir errores o experimentar con cambios, sin poner en riesgo la estabilidad del código principal. Gracias a que las ramas en Git son ligeras y muy económicas de crear, es una práctica común generar una rama nueva para cada nueva tarea o característica.

4. ¿Cómo saber en cuál rama estoy trabajando?

La forma más directa de saber en qué rama se está trabajando es ejecutar el comando **git branch**, el cual muestra la lista de ramas locales del repositorio y marca con un asterisco (*) la rama activa actualmente. También se puede usar el comando más moderno **git status**, cuya primera línea de salida indica explícitamente el nombre de la rama en la que se encuentra el usuario en ese momento (por ejemplo: "On branch main").

Adicionalmente, muchas terminales y sistemas de shell (como Bash o Zsh con complementos, o entornos como Oh My Zsh) muestran el nombre de la rama activa directamente en el prompt de la línea de comandos, lo que facilita identificarla sin necesidad de ejecutar comandos adicionales. Los editores de código y entornos de desarrollo integrados (IDE) como VS Code también muestran la rama actual en la barra inferior de la interfaz.

5. ¿Quién creó Git?

Git fue creado por **Linus Torvalds**, el mismo desarrollador finlandés-estadounidense conocido por haber creado el núcleo (kernel) de Linux. Torvalds desarrolló Git en el año 2005 como respuesta a una necesidad muy concreta: el sistema propietario que utilizaba la comunidad del kernel de Linux para el control de versiones (BitKeeper) dejó de estar disponible de forma gratuita para ese proyecto.

Ante esta situación, Torvalds decidió crear una herramienta propia, con el objetivo de que fuera rápida, distribuida, capaz de manejar proyectos muy grandes (como el propio kernel de Linux) de forma eficiente, y que garantizara la integridad de los datos frente a corrupciones o alteraciones maliciosas. Con el tiempo, el mantenimiento de Git pasó a manos de otros colaboradores, entre ellos Junio Hamano, quien ha sido el mantenedor principal del proyecto desde poco después de su creación.

6. ¿Cuáles son los comandos esenciales de Git?

Existe un conjunto de comandos básicos que conforman el flujo de trabajo cotidiano con Git. Los más esenciales son:

- **git init**: inicializa un nuevo repositorio Git en el directorio actual.
- **git clone [url]**: descarga una copia completa de un repositorio remoto existente.
- **git status**: muestra el estado actual del repositorio (archivos modificados, añadidos, rama actual, etc.).
- **git add [archivo]**: agrega cambios al área de preparación (staging area).
- **git commit -m "mensaje"**: guarda los cambios preparados como un nuevo punto en el historial del proyecto.
- **git branch**: lista, crea o elimina ramas.
- **git checkout [rama]** / **git switch [rama]**: cambia entre ramas existentes.
- **git merge [rama]**: fusiona los cambios de una rama con la rama activa.
- **git pull**: descarga y combina los cambios del repositorio remoto con el local.
- **git push**: envía los commits locales al repositorio remoto.
- **git log**: muestra el historial de commits del repositorio.
- **git diff**: muestra las diferencias entre archivos o versiones.

Dominar estos comandos permite cubrir el ciclo completo de trabajo con Git: crear o clonar un repositorio, registrar cambios, trabajar en paralelo mediante ramas, y sincronizar el trabajo con un servidor remoto compartido.

7. ¿Qué es Git Flow?

Git Flow es un modelo o estrategia de ramificación (branching model) propuesto por Vincent Driessen en 2010, que define un conjunto de reglas claras sobre cómo deben crearse, nombrarse y combinarse las ramas dentro de un proyecto. Su objetivo es ordenar el desarrollo de software en proyectos con ciclos de lanzamiento (releases) bien definidos.

El modelo se basa en el uso de dos ramas principales de vida larga: **main** (o master), que refleja siempre el código en producción, y **develop**, que contiene los últimos cambios integrados para la siguiente versión. A partir de estas se derivan ramas de apoyo temporales y específicas:

- **feature/***: para desarrollar nuevas funcionalidades, se crean desde develop y se fusionan de vuelta a develop.
- **release/***: para preparar una nueva versión antes de publicarla (ajustes finales, pruebas, corrección de errores menores).
- **hotfix/***: para corregir errores críticos directamente sobre producción, sin esperar al siguiente ciclo de release.

Git Flow resulta útil en proyectos con lanzamientos programados y versiones bien diferenciadas, aunque suele considerarse más complejo y pesado para equipos que practican integración y entrega continua (CI/CD), donde modelos más simples como Trunk Based Development suelen preferirse.

8. ¿Qué es el desarrollo basado en trunk (Trunk Based Development)?

El desarrollo basado en trunk (Trunk Based Development, TBD) es una práctica de control de versiones en la que todos los desarrolladores integran sus cambios de manera frecuente —normalmente varias veces al día— directamente sobre una única rama principal, llamada “trunk” (equivalente a main o master), en lugar de mantener ramas de larga duración para cada funcionalidad.

En este enfoque, las ramas, cuando se utilizan, tienen una vida muy corta (horas o, como máximo, un par de días) y se fusionan rápidamente de vuelta al trunk. Para evitar que funcionalidades incompletas afecten la rama principal, es común apoyarse en técnicas como los **feature flags** (banderas de características), que permiten integrar código inacabado o deshabilitado sin exponerlo a los usuarios finales hasta que esté listo.

Trunk Based Development es una práctica central dentro de la integración continua (CI) y la entrega continua (CD), ya que minimiza los conflictos de fusión, evita el llamado “merge hell” (grandes y complicadas fusiones acumuladas), y favorece ciclos de retroalimentación más rápidos, pruebas automatizadas constantes y despliegues más frecuentes y confiables.

Bibliografía

Chacon, S., & Straub, B. (2014). *Pro Git* (2.^a ed.). Apress. Disponible en: <https://git-scm.com/book/en/v2>

Git. (s.f.). *Git — Documentación oficial*. Software Freedom Conservancy. <https://git-scm.com/doc>

Driessen, V. (2010). *A successful Git branching model*.
<https://nvie.com/posts/a-successful-git-branching-model/>

Atlassian. (s.f.). *Git tutorials and workflows*. Atlassian Bitbucket. <https://www.atlassian.com/git/tutorials>

GitHub. (s.f.). *Trunk-based development*. GitHub Resources. <https://resources.github.com/>

Torvalds, L. (2005). *Git — Historia y creación del proyecto*. Linux Kernel Mailing List (archivo histórico).
<https://lore.kernel.org/>